



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A++' Grade

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Lab Manual

Course: Programming with Java

SEM-III

A.Y. 2026-2027



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A++' Grade

Vision

To evolve as a center of excellence in Computer Science and Engineering to produce skilled and proficient global professionals to build the society.

Mission

- To provide the conducive environment for establishing students in the global platform of research and innovation.
- To educate students on cutting edge technologies with problem solving capabilities, leadership and teamwork skills.
- To inculcate the professional values with lifelong learning through curricular and co-curricular activities and create globally-aware disciplined citizens.
- To commence various initiatives for motivating students to work for the betterment of society.

Program Outcomes

1. Engineering Knowledge: Math, science, engineering fundamentals applied to Complex Engineering Problems
2. Problem Analysis Identify, formulate, and analyse complex problems — with explicit attention to the holistic nature of the problem
3. Design / Development of Solutions Solutions must explicitly consider public health, safety, cultural, societal, and environmental considerations — sustainability is now intrinsic to Design
4. Conduct Investigations of Complex Problems Research methodology, experimentation, analysis, interpretation of data — with sustainability woven in
5. Engineering Tool Usage Modern engineering tools, software, prediction and modelling techniques. Digital literacy is now emphasised across all Pos
6. The Engineer and the World Societal context, environmental context, sustainability, and contemporary issues understood and integrated
7. Ethics Expanded under GAPC v4.0 to include human values, diversity and inclusion beyond pure professional ethics



8. Individual and Collaborative Team Work Function effectively as an individual, in teams, in diverse settings
9. Communication Written, oral, technical communication; engineering activities reporting
10. Project Management & Finance Engineering and management principles applied to projects, multidisciplinary environments
11. Life-Long Learning Recognise the need for, and engage in, independent and life-long learning

Course Outcomes

CO1	Describe and implement program on the basic concepts of Java programming language.
CO2	Apply concepts of inheritance and interface to the given real-world problem.
CO3	Implement error handling techniques using exception handling.
CO4	Demonstrate database connectivity using JDBC.
CO5	Explain multithreading and networking concept with RMI and socket programming

Articulation Matrix:

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2
CO1	3	2	1	-	3	-	-	-	-	-	-	2	-
CO2	3	3	3	-	2	-	-	1	-	-	-	2	-
CO3	2	3	2	1	2	-	-	-	-	-	-	2	-
CO4	2	2	2	1	3	-	-	-	-	-	-	2	-
CO5	3	2	2	2	3	-	-	2	-	-	-	2	-



Assignment No	Assignment Name	CO
1.	Install and configure the Java environment, then develop and execute a Java program using variables and methods through the command-line interface without using an IDE.	CO1
2.	Develop a Java application demonstrating the use of default, parameterized, and copy constructors for object initialization and data management.	CO1
3.	Develop a Java application demonstrating method overloading and the use of static variables, methods in class design.	CO1
4.	Develop a Java application demonstrating the use of wrapper classes for conversion between primitive data types and objects, and performing basic operations on wrapped values.	CO1
5.	Develop a Java application demonstrating the use of ArrayList, Vector, and StringBuffer for dynamic data storage, management, and string manipulation.	CO1
6.	Write a Java program to demonstrate the use of an inner class, an anonymous class.	CO1
7.	Implement a Java program to demonstrate inheritance and interfaces using suitable real-world examples.	CO2
8.	Write a Java program to demonstrate the use of the super keyword in inheritance.	CO2
9.	Write a Java program to demonstrate the use of the final keyword with variables, methods, and classes.	CO2
10.	Write a Java program to demonstrate the use of abstract classes and abstract methods	CO2
11.	Write a Java program to demonstrate the implementation of interfaces.	CO2
12.	Write a Java program to create and use user-defined packages.	CO2
13.	Write a Java program to perform reading from and writing to a file using file handling operations.	CO2
14.	Write a Java program to demonstrate exception handling using try and catch blocks, nested and multiple try-catch blocks.	CO3



15.	Write a Java program to demonstrate exception handling using finally, throw, and throws	CO3
16.	Write a Java program to create and handle a user-defined exception.	CO3
17.	Write a Java program to demonstrate the use of Swing and Applets	CO3
18.	Write a Java GUI program to demonstrate button click event handling	CO3
19.	Develop a Java application to establish connectivity with a database and perform data retrieval using the SELECT operation.	CO4
20.	Develop a Java application to establish connectivity with a database and perform INSERT, UPDATE, and DELETE operations.	CO4
21.	Demonstration and exploration of JDBC classes and interfaces (DriverManager, Connection, Statement).	CO4
22.	Demonstration and exploration of JDBC classes and interfaces (PreparedStatement, and ResultSet).	CO4
23.	Develop a Java application using JDBC to navigate and retrieve records from a ResultSet.	CO4
24.	Develop a Java GUI application integrated with JDBC to perform database operations."	CO4
25.	Write a Java program using the Thread class to create multiple threads and demonstrate their lifecycle.	CO5
26.	Write a Java program to create and execute multiple threads using the Runnable interface.	CO5
27.	Develop a Java application demonstrating socket programming for communication between a client and a server.	CO5
28.	Develop a Java application demonstrating Remote Method Invocation (RMI) using stub and skeleton components for remote communication.	CO5
29.	Write a JSP application to demonstrate the basic concepts of Java Server Pages.	CO5
30.	Write a JSP application to demonstrate CRUD operations.	CO5



ASSIGNMENT No: 1

TITLE: Develop an application based on Java environment setup, JDK/JRE/JVM, variables, methods and constructors. and demonstrate the implementation of the concepts through suitable examples.

Problem Statement

Develop an application based on Java environment setup, JDK/JRE/JVM, variables, methods and constructors. and demonstrate the implementation of the concepts through suitable examples.

Learning Objectives

Understand the theoretical concepts, implement them in Java programs, analyze outputs, and apply the concepts to solve practical programming problems.

Theory

Java environment setup, JDK/JRE/JVM, variables, methods and constructors. forms an important component of Java programming. Students should understand the underlying concepts, architecture, advantages, use cases, and implementation methodology. The exercise should focus on understanding the design principles, coding practices, execution flow, and practical applications of these concepts in software development. Detailed study should include syntax, APIs, execution process, and real-world applications.

Exercise

1. Explain the main concept used in Java environment setup, variables and methods.
2. What are the advantages of using Java variables and methods?
3. How can this implementation be improved?
4. What are the real-world applications?



ASSIGNMENT No: 2

TITLE: Develop a Java application demonstrating the use of default, parameterized, and copy constructors for object initialization and data management.

Problem Statement

Develop a Java application demonstrating the use of default, parameterized, and copy constructors for object initialization and data management.

Learning Objectives

To understand and implement default, parameterized, and copy constructors for object initialization.

Theory

Constructors are special member functions that are automatically invoked when an object is created. Java supports default constructors provided by the compiler, parameterized constructors for initializing objects with user-defined values, and copy constructors (implemented manually) for creating duplicate objects. Constructors improve code readability and simplify object initialization while supporting object-oriented programming principles.

Exercise

1. Create a **Student class** using default and parameterized constructors to initialize student name and roll number.
2. Develop a **Mobile Phone Inventory System** using different constructors to initialize mobile details and create duplicate object records.



ASSIGNMENT No: 3

TITLE: Develop a Java application demonstrating method overloading and the use of static variables, methods in class design.

Problem Statement

Develop a Java application demonstrating method overloading and the use of static variables, methods in class design.

Learning Objectives

To implement method overloading and use static variables and methods in Java programs.

Theory

Method overloading allows multiple methods to have the same name with different parameter lists, enabling compile-time polymorphism. The static keyword is used for variables and methods that belong to the class instead of individual objects. Static members share common data among all objects and reduce memory usage. These concepts improve code flexibility, readability, and efficient resource management.

Exercise

1. Develop a **Calculator program** using overloaded methods for addition of integers and decimals. Use a static variable to count calculations.
2. Develop a **Restaurant Billing Application** where overloaded methods calculate bills for dine-in, takeaway, and delivery orders, while static variables track total orders.



ASSIGNMENT No: 4

TITLE: Develop a Java application demonstrating the use of wrapper classes for conversion between primitive data types and objects, and performing basic operations on wrapped values.

Problem Statement

Develop a Java application demonstrating the use of wrapper classes for conversion between primitive data types and objects, and performing basic operations on wrapped values.

Learning Objectives

To perform conversion between primitive data types and wrapper objects using Java wrapper classes.

Theory

Wrapper classes are predefined classes that encapsulate primitive data types into objects. Java provides wrapper classes such as Integer, Double, Character, and Boolean for every primitive type. They support autoboxing, unboxing, parsing, and conversion methods. Wrapper classes are widely used in Java Collections Framework and when object representation of primitive values is required.

Exercise

1. Create a program to convert student marks from String format to Integer and calculate total marks.
2. Develop an **Employee Payroll System** that accepts employee IDs, basic salary, and bonus amounts from the user. Convert the entered values into wrapper objects and perform validation operations to ensure valid salary values before calculating the net salary.



ASSIGNMENT No: 5

TITLE: Develop a Java application demonstrating the use of ArrayList, Vector, and StringBuffer for dynamic data storage, management, and string manipulation.

Problem Statement

Develop a Java application demonstrating the use of ArrayList, Vector, and StringBuffer for dynamic data storage, management, and string manipulation.

Learning Objectives

To implement dynamic data storage and string manipulation using ArrayList, Vector, and StringBuffer.

Theory

ArrayList and Vector are dynamic array implementations provided by the Java Collections Framework. ArrayList offers faster performance in single-threaded applications, whereas Vector provides synchronized operations for thread safety. StringBuffer is a mutable sequence of characters that allows efficient string modification without creating new objects. These classes are commonly used in applications requiring dynamic memory allocation and efficient string processing.

Exercise

1. Create a **To-Do List application** using ArrayList to store tasks and StringBuffer to display tasks.
2. Create a **Student Course Registration System** using ArrayList to store the list of courses registered by a student and StringBuffer to generate and display the registered course list. The application should allow users to add, remove, and view registered courses.



ASSIGNMENT No: 6

TITLE: Write a Java program to demonstrate the use of an inner class, an anonymous class.

Problem Statement

Write a Java program to demonstrate the use of an inner class, an anonymous class.

Learning Objectives

To implement inner classes and anonymous classes for improving encapsulation and event handling in Java.

Theory

An inner class is a class defined inside another class and has access to all members of the enclosing class, including private members. Inner classes help improve code organization and logical grouping of related classes. Anonymous classes are unnamed inner classes that are instantiated only once and are mainly used for implementing interfaces or abstract classes. They simplify code by eliminating the need for separate class definitions. These concepts are widely used in Java GUI programming and event handling.

Outcome

Conclusion

Exercise

1. Create a **Vehicle program** where an inner class displays vehicle details and anonymous class performs an action.
2. Develop a **Food Delivery Application** where an inner class handles order details and anonymous classes handle delivery status updates.



ASSIGNMENT No: 7

TITLE: Implement a Java program to demonstrate inheritance and interfaces using suitable real-world examples.

Problem Statement

Implement a Java program to demonstrate inheritance and interfaces using suitable real-world examples.

Learning Objectives

To implement inheritance and interfaces for achieving code reusability and abstraction.

Theory

Inheritance allows one class to acquire the properties and methods of another class, promoting code reuse and reducing redundancy. Java supports single inheritance through the extends keyword. Interfaces define a contract that implementing classes must follow, enabling abstraction and multiple inheritance of type. Together, inheritance and interfaces support polymorphism and modular software development. They are fundamental concepts of object-oriented programming.

Exercise

1. Create a **Shape application** where Circle and Rectangle inherit from Shape and calculate area.
2. Develop a **E-Commerce Product System** where electronic, clothing, and grocery products inherit common properties and implement product interfaces.



ASSIGNMENT No: 8

TITLE: Write a Java program to demonstrate the use of the super keyword in inheritance.

Problem Statement

Write a Java program to demonstrate the use of the super keyword in inheritance.

Learning Objectives

To demonstrate the use of the super keyword for accessing parent class members.

Theory

The super keyword is used to refer to the immediate parent class in Java. It can invoke parent class constructors, access parent class variables, and call overridden methods. Super helps distinguish between parent and child class members with the same name. It ensures proper constructor chaining during object creation. It is an important concept in inheritance and method overriding.

Exercise

1. Create an **Employee and Manager program** where Manager accesses Employee details using super keyword.
2. Develop a **Vehicle Insurance System** where child insurance classes access parent vehicle information using the super keyword.



ASSIGNMENT No: 9

TITLE: Write a Java program to demonstrate the use of the final keyword with variables, methods, and classes.

Problem Statement

Write a Java program to demonstrate the use of the final keyword with variables, methods, and classes.

Learning Objectives

To implement the final keyword with variables, methods, and classes.

Theory

The final keyword restricts modification in Java programs. A final variable cannot be reassigned after initialization. A final method cannot be overridden by subclasses, while a final class cannot be inherited. This improves security, prevents accidental modifications, and increases program reliability. Final is commonly used for constants and immutable classes.

Exercise

1. Create a **Bank Account program** where account number is final and cannot be changed.
2. Create a **Library Book Management** program where the **book ISBN** is declared as **final** and cannot be changed once assigned. Display the book's ISBN, title, author, and price.



ASSIGNMENT No: 10

TITLE: Write a Java program to demonstrate the use of abstract classes and abstract methods

Problem Statement

Write a Java program to demonstrate the use of abstract classes and abstract methods

Learning Objectives

To implement abstraction using abstract classes and abstract methods.

Theory

Abstract classes provide partial implementation of a program and cannot be instantiated directly. They may contain both abstract and concrete methods. Abstract methods declare functionality that must be implemented by subclasses. Abstraction hides implementation details while exposing only essential functionality. This improves code flexibility and maintainability in object-oriented programming.

Exercise

1. Create an abstract **Payment class** and implement Credit Card and UPI payment classes.?
2. Create an abstract **FoodOrder** class with an abstract method `calculateBill()`. Implement two subclasses, **DineInOrder** and **TakeAwayOrder**, to calculate and display the total bill based on the order type.



ASSIGNMENT No: 11

TITLE: Write a Java program to demonstrate the implementation of interfaces.

Problem Statement

Write a Java program to demonstrate the implementation of interfaces.

Learning Objectives

To implement interfaces for achieving abstraction and multiple inheritance of type.

Theory

An interface specifies a set of abstract methods that implementing classes must define. It provides complete abstraction and supports multiple inheritance of type. Interfaces improve modularity, loose coupling, and software flexibility. They allow different classes to implement common behavior while maintaining independent implementations. Interfaces are widely used in enterprise application development.

Exercise

1. Create an interface **Printable** and implement it in Student and Employee classes.
2. Create an interface **Switchable** with a method turnOn(). Implement the interface in **Light** and **Fan** classes to display the device status.



ASSIGNMENT No: 12

TITLE: Write a Java program to create and use user-defined packages.

Problem Statement

Write a Java program to create and use user-defined packages.

Learning Objectives

To create and use user-defined packages for organizing Java programs.

Theory

Packages are used to organize related classes and interfaces into namespaces. User-defined packages improve code organization, reduce naming conflicts, and enhance reusability. They also support access protection and modular application development. Java packages simplify project management by grouping logically related components together. Large-scale software systems extensively use packages.

Exercise

1. Create a Java program by developing user-defined packages for a College Management System. Create separate packages for student and faculty details, and display their information using classes from these packages.
2. Create a Java program by developing a **user-defined package** named library that contains a class to store and display book details such as book ID, title, author, and price. Import the package into the main program and display the book information.



ASSIGNMENT No: 13

TITLE: Write a Java program to perform reading from and writing to a file using file handling operations.

Problem Statement

Write a Java program to perform reading from and writing to a file using file handling operations.

Learning Objectives

To perform file reading and writing operations using Java file handling classes.

Theory

Java provides file handling through the java.io package. Classes such as File, FileReader, FileWriter, BufferedReader, and BufferedWriter are used to perform input and output operations. File handling enables permanent storage and retrieval of application data. Proper exception handling is necessary to manage file-related errors. These concepts are essential for data persistence.

Exercise

1. Create a program to write and read **employee details from a file**.
2. Create a Java program to write **patient details** (patient ID, name, age, and diagnosis) to a file and read the details from the file for display.



ASSIGNMENT No: 14

TITLE: Write a Java program to demonstrate exception handling using try and catch blocks, nested and multiple try-catch blocks.

Problem Statement

Write a Java program to demonstrate exception handling using try and catch blocks, nested and multiple try-catch blocks.

Learning Objectives

To handle runtime exceptions using try-catch, nested try, and multiple catch blocks.

Theory

Exception handling prevents abnormal termination of programs due to runtime errors. The try block contains code that may generate exceptions, while catch blocks handle specific exceptions. Nested and multiple try-catch structures allow complex error management. Exception handling improves software reliability and user experience by gracefully managing unexpected situations.

Exercise

1. Create an ATM program handling invalid withdrawal amount using try-catch
2. Create an Online Shopping program that handles an invalid product quantity entered by the user using a try-catch block. Display an appropriate error message if the quantity is less than or equal to zero.



ASSIGNMENT No: 15

TITLE: Write a Java program to demonstrate exception handling using finally, throw, and throws.

Problem Statement

Write a Java program to demonstrate exception handling using finally, throw, and throws.

Learning Objectives

To implement finally, throw, and throws for effective exception management.

Theory

The finally block always executes regardless of whether an exception occurs, making it suitable for resource cleanup. The throw keyword explicitly generates an exception, while throws declares exceptions that a method may pass to its caller. These mechanisms improve structured error handling and application stability.

Exercise

1. Create a Login program that throws an exception for invalid password and uses finally block.
2. Create an **ATM PIN Verification** program that throws an exception for an invalid PIN entered by the user and uses a **finally** block to display a message indicating that the verification process has completed.



ASSIGNMENT No: 16

TITLE: Write a Java program to create and handle a user-defined exception.

Problem Statement

Write a Java program to create and handle a user-defined exception.

Learning Objectives

To create and implement custom exceptions for application-specific error handling.

Theory:

User-defined exceptions are created by extending the Exception class. They allow developers to define meaningful exceptions for specific business logic. Custom exceptions improve program readability and simplify debugging by providing descriptive error messages. They are widely used in enterprise software and validation systems.

Exercise

1. Create a **Voting System** that throws a custom exception if age is below 18
2. Create a **Driving License System** that throws a **custom exception** if the user's age is below 18. If the age is valid, display that the user is eligible for a driving license



ASSIGNMENT No: 17

TITLE: Write a Java program to demonstrate the use of Swing and Applets

Problem Statement

Write a Java program to demonstrate the use of Swing and Applets

Learning Objectives

To develop graphical user interfaces using Swing components and Applets.

Theory

Swing is Java's GUI toolkit that provides lightweight components such as buttons, labels, text fields, and menus. Applets were small Java programs embedded in web pages for interactive applications. Although Applets are obsolete, Swing remains an important framework for desktop application development. Swing supports platform-independent graphical interfaces.

Exercise

1. Create a simple **Student Registration Form GUI** using Swing.
2. Create a simple **Employee Registration Form** using **Java Swing GUI** that takes employee details such as Employee ID, Name, Department, and Salary. Display the entered information in a dialog box



ASSIGNMENT No: 18

TITLE: Write a Java GUI program to demonstrate button click event handling.

Problem Statement

Write a Java GUI program to demonstrate button click event handling.

Learning Objectives

To implement button click events using Java event handling mechanisms.

Theory

Event handling allows Java programs to respond to user actions such as button clicks, mouse movements, and keyboard input. Swing uses ActionListener to process button click events. Event-driven programming separates user interface design from event-processing logic, making applications interactive and user-friendly.

Exercise

1. Create a GUI calculator where buttons perform addition and subtraction.
2. Create a **Bank Balance Calculator GUI application** using Java Swing where the user enters the initial balance and transaction amount. Use buttons to perform **addition (deposit)** and **subtraction (withdrawal)** and display the updated balance.



ASSIGNMENT No: 19

TITLE: Develop a Java application to establish connectivity with a database and perform data retrieval using the SELECT operation.

Problem Statement

Develop a Java application to establish connectivity with a database and perform data retrieval using the SELECT operation.

Learning Objectives

To establish database connectivity and retrieve records using JDBC SELECT statements.

Theory

Java Database Connectivity (JDBC) is a standard Java API that enables communication between Java applications and relational databases. It provides a platform-independent mechanism to connect to databases such as MySQL, Oracle, PostgreSQL, and SQL Server. Before executing SQL statements, a JDBC driver is loaded and a connection is established using the DriverManager class. The Connection object represents an active connection to the database, while the Statement object is used to execute SQL queries. The **SELECT** statement retrieves records from database tables based on specified conditions. The retrieved records are stored in a ResultSet object, which allows sequential access to the data. JDBC simplifies database programming and is widely used in desktop, web, and enterprise applications.

Exercise

1. Create a Java application to display **student records from database** using SELECT query
2. Create a Java application that connects to a database and displays **product details** using a **SELECT query**. Show product ID, product name, quantity, and price.



ASSIGNMENT No: 20

TITLE: Develop a Java application to establish connectivity with a database and perform INSERT, UPDATE, and DELETE operations.

Problem Statement

Develop a Java application to establish connectivity with a database and perform INSERT, UPDATE, and DELETE operations.

Learning Objectives

To perform database modification operations using JDBC.

Theory

JDBC allows Java applications to perform database manipulation using SQL Data Manipulation Language (DML) statements such as INSERT, UPDATE, and DELETE. The INSERT statement adds new records into database tables, UPDATE modifies existing records, and DELETE removes unwanted records. These operations are executed using the Statement or PreparedStatement interface. PreparedStatement is generally preferred because it supports parameterized queries, improves performance, and prevents SQL injection attacks. Proper transaction management ensures that database modifications are performed safely and consistently. These operations form the basis of almost every database-driven application such as banking systems, hospital management systems, and student information systems.

Exercise

1. Create a Java application to manage **employee records** using CRUD operations
2. Create a Java application to manage **student records** using **CRUD operations**. Perform Create, Read, Update, and Delete operations on student details such as roll number, name, course, and marks using a database.



ASSIGNMENT No: 21

TITLE: Demonstration and exploration of JDBC classes and interfaces (DriverManager, Connection, Statement).

Problem Statement

Demonstration and exploration of JDBC classes and interfaces (DriverManager, Connection, Statement).

Learning Objectives

To understand and use the core JDBC classes for database connectivity.

Theory

The JDBC API consists of several classes and interfaces that facilitate database connectivity. The DriverManager class is responsible for loading the appropriate JDBC driver and establishing a connection with the database using a database URL, username, and password. The Connection interface represents an active session between the Java application and the database server. It provides methods to create SQL statements and manage transactions. The Statement interface executes static SQL queries and updates. These classes work together to establish communication between Java programs and databases. Understanding these core components is essential for developing database applications using Java.

Exercise

1. Create a program to connect Java with database and display database connection status.
2. Create a Java program to connect with a database and display the **connection status**. After successful connection, show a message indicating that the **Student database is connected successfully**.



Subject: Programming with Java

ASSIGNMENT No: 22

TITLE: Demonstration and exploration of JDBC classes and interfaces (PreparedStatement, and ResultSet).

Problem Statement

Demonstration and exploration of JDBC classes and interfaces (PreparedStatement, and ResultSet).

Learning Objectives

To implement PreparedStatement and ResultSet for secure query execution and data retrieval.

Theory

PreparedStatement is an advanced JDBC interface used to execute precompiled SQL statements with parameters. Unlike the Statement interface, PreparedStatement allows SQL queries to be compiled only once and executed multiple times with different values, thereby improving performance. It also protects applications against SQL injection attacks by separating SQL code from user input. The ResultSet interface stores the records returned by SQL SELECT queries. It provides various methods such as next(), getInt(), getString(), and getDouble() for accessing retrieved data. PreparedStatement and ResultSet together provide secure, efficient, and flexible database interaction and are widely used in enterprise Java applications.

Exercise

1. Create a login application using PreparedStatement to check username and password.
2. Create a Java application using **PreparedStatement** to authenticate **hospital staff login** (doctor/nurse/login ID and password). Display appropriate access messages based on authentication result.



ASSIGNMENT No: 23

TITLE: Develop a Java application using JDBC to navigate and retrieve records from a ResultSet.

Problem Statement

Develop a Java application using JDBC to navigate and retrieve records from a ResultSet.

Learning Objectives

To navigate and retrieve records using ResultSet methods.

Theory

A ResultSet object contains data retrieved from a database after executing a SELECT query. It acts like a cursor that points to one row of the result at a time. Initially, the cursor is positioned before the first row. Various methods such as next(), previous(), first(), last(), absolute(), and relative() allow movement through the records. Data from each column can be retrieved using methods like getString(), getInt(), and getDouble(). Scrollable and updatable ResultSets provide greater flexibility for navigating and modifying retrieved data. ResultSet plays a crucial role in displaying and processing database information in Java applications.

Exercise

1. Create a program to display database records one by one using ResultSet.
2. Create a Java program that retrieves **employee records from a database** and displays them **one by one using ResultSet**. Show employee ID, name, department, and salary sequentially



ASSIGNMENT No: 24

TITLE: Develop a Java GUI application integrated with JDBC to perform database operations."

Problem Statement

Develop a Java GUI application integrated with JDBC to perform database operations."

Learning Objectives

To integrate Java GUI applications with databases using JDBC.

Theory

Java Swing provides graphical user interface components such as buttons, labels, text fields, tables, and forms, while JDBC enables database connectivity. Combining Swing with JDBC allows developers to create interactive desktop applications capable of performing database operations through a graphical interface. Users can enter information using GUI components, which is then processed and stored in the database using SQL queries. Retrieved records can be displayed in tables or text fields for easy viewing and modification. This integration improves user experience by eliminating command-line interaction and forms the foundation of desktop applications such as inventory management systems, library management systems, and student management systems.

Exercise

1. Create a **Library Management System GUI** connected with database
2. Create a **Book Issue Tracking System GUI application** using Java Swing connected to a database. The system should store book ID, student name, issue date, and return date, and allow record management.



ASSIGNMENT No: 25

TITLE: Write a Java program using the Thread class to create multiple threads and demonstrate their lifecycle.

Problem Statement

Write a Java program using the Thread class to create multiple threads and demonstrate their lifecycle.

Learning Objectives

To create and manage multiple threads using the Thread class.

Theory

A thread is the smallest unit of execution within a Java program. Multithreading enables multiple tasks to execute concurrently, improving application performance and responsiveness. Java provides the Thread class for creating threads by extending it and overriding the run() method. A thread passes through different lifecycle stages including New, Runnable, Running, Blocked/Waiting, and Terminated. The scheduler manages thread execution based on system resources. Methods such as start(), sleep(), join(), and interrupt() control thread behavior. Multithreading is commonly used in games, web servers, animation, and background processing applications.

Exercise

1. Create a program where multiple threads print numbers simultaneously
2. Create a Java program using **multiple threads** to simulate bank transactions. One thread performs **deposit operations**, and another performs **withdrawal operations**, both printing transaction details simultaneously.



ASSIGNMENT No: 26

TITLE: Write a Java program to create and execute multiple threads using the Runnable interface.

Problem Statement

Write a Java program to create and execute multiple threads using the Runnable interface.

Learning Objectives

To create multiple threads using the Runnable interface.

Theory

The Runnable interface provides an alternative approach to creating threads in Java. A class implementing Runnable defines the run() method that contains the code executed by the thread. The Runnable object is passed to a Thread object, which starts execution using the start() method. Unlike extending the Thread class, implementing Runnable allows a class to inherit from another superclass, making the design more flexible. Runnable separates task definition from thread execution, resulting in better code organization and reusability. It is widely used in multithreaded enterprise applications and thread pool implementations.

Exercise

1. Create a program using Runnable to run two tasks simultaneously.
2. Create a Java program using the **Runnable interface** where two tasks run simultaneously: one thread performs **deposit operations**, and another performs **withdrawal operations**, both displaying results concurrently.



ASSIGNMENT No: 27

TITLE: Develop a Java application demonstrating socket programming for communication between a client and a server.

Problem Statement

Develop a Java application demonstrating socket programming for communication between a client and a server.

Learning Objectives

To establish communication between client and server using Java sockets.

Theory

Socket programming enables communication between computers over a network using TCP/IP protocols. Java provides the Socket class for client-side communication and the ServerSocket class for server-side communication. A server listens for client requests on a specific port, while clients establish connections using the server's IP address and port number. Once connected, data is exchanged using input and output streams. Socket programming supports reliable communication between distributed systems and is widely used in chat applications, online gaming, file transfer systems, remote monitoring, and client-server architectures.

Exercise

1. Create a simple **client-server chat application** using sockets.
2. Create a Java program using **socket programming** where the client sends a message to the server, and the server returns the **same message back (echo system)**.



ASSIGNMENT No: 28

TITLE: Develop a Java application demonstrating Remote Method Invocation (RMI) using stub and skeleton components for remote communication.

Problem Statement

Develop a Java application demonstrating Remote Method Invocation (RMI) using stub and skeleton components for remote communication.

Learning Objectives

To implement Remote Method Invocation for distributed Java applications.

Theory

Remote Method Invocation (RMI) is a Java technology that enables an object running on one Java Virtual Machine (JVM) to invoke methods of an object running on another JVM located on a different machine. RMI simplifies distributed application development by allowing remote objects to behave like local objects. It consists of remote interfaces, remote object implementations, the RMI registry, and client-server communication. The registry maintains references to remote objects, allowing clients to locate and invoke them. RMI provides platform-independent communication, object serialization, and automatic network handling, making it suitable for distributed enterprise applications.

Exercise

1. Create a remote calculator service using Java RMI
2. Create a Java application using **RMI** where the server converts temperature values between **Celsius and Fahrenheit**. The client sends input temperature and receives converted output.



ASSIGNMENT No: 29

TITLE: Write a JSP application to demonstrate the basic concepts of Java Server Pages.

Problem Statement

Write a JSP application to demonstrate the basic concepts of Java Server Pages.

Learning Objectives

To develop dynamic web pages using Java Server Pages.

Theory

Java Server Pages (JSP) is a server-side technology used for developing dynamic web applications. JSP allows Java code to be embedded within HTML pages using special JSP tags. When a JSP page is requested, the web server automatically translates it into a servlet, compiles it, and executes it to generate dynamic web content. JSP supports implicit objects, directives, scripting elements, expression language, and standard tag libraries. It simplifies web development by separating presentation from business logic. JSP is commonly used with Servlets and JDBC for building database-driven web applications.

Exercise

1. Create a JSP page to display student information entered by the user
2. Create a **JSP application** that takes user registration details such as Name, Email, Phone Number, and Password, and displays the submitted information on a result JSP page.



ASSIGNMENT No: 30

TITLE: Write a JSP application to demonstrate CRUD operations.

Problem Statement

Write a JSP application to demonstrate CRUD operations.

Learning Objectives

To implement Create, Read, Update, and Delete operations using JSP and a database.

Theory

CRUD stands for Create, Read, Update, and Delete, which represent the four basic database operations performed in web applications. JSP works together with Servlets and JDBC to implement these operations. Users interact with HTML forms created using JSP, while JDBC executes SQL statements to manipulate database records. INSERT adds new records, SELECT retrieves information, UPDATE modifies existing data, and DELETE removes unwanted records. Proper validation, exception handling, and session management improve application reliability and security. JSP CRUD applications are widely used in employee management systems, online shopping portals, hospital management systems, banking applications, and other enterprise software.

Exercise

1. Create a JSP-based application to add, update, delete, and view employee details
2. Create a **JSP-based application** to manage **patient records** using **CRUD operations**. Include fields such as patient ID, name, age, disease, and treatment status.